BABU BANARASI DAS UNIVERSITY

2026-2027

SCHOOL OF COMPUTER APPLICATIONS

Machine Learning Project On

Food and Customer Demand Prediction

Submitted to: Mr. Harendra Singh

Department: BCA (DS & AI)

Submitted by:

Ashutosh Pandey

Roll no: 1250258147

FOOD AND CUSTOMER DEMAND PREDICTION

Machine Learning Project Notebook (food.ipynb)

Data Analysis, Visualisation and Model Building

ACKNOWLEDGEMENT

I sincerely express my gratitude to my teacher Mr. Harendra Singh for giving me the opportunity to work on this machine learning project titled “Food and Customer Demand Prediction”.

This project helped me understand how to clean and explore a real-world dataset, visualise it, and build regression models using Python libraries such as Pandas, Matplotlib, Seaborn and Scikit-learn. I also learned how different models are trained, tuned with GridSearchCV and compared using the R^2 score.

I am thankful to my teacher for the guidance and support provided throughout the completion of this project.

INTRODUCTION

Restaurants and food outlets must decide every day how much food to prepare. Preparing too little causes lost sales and unhappy customers, while preparing too much leads to food waste and financial loss. Machine learning can help by predicting demand from past data.

In this project, a food demand dataset with 8,563 records (after removing missing values) and 25 original columns was analysed. It contains information such as day, month, weekend, holiday, meal type, weather, temperature, food category, price, promotion, expected customers, previous day and previous week sales, expected demand, prepared quantity, expected waste and waste risk.

The data was cleaned by dropping missing values, categorical columns such as Weather and Meal_Type were converted to numeric labels, and the data was explored using correlation heatmaps, count plots and pie charts. Regression models were then built to predict Expected_Demand and Expected_Customers.

OBJECTIVES

- To load and clean the food demand dataset and handle missing values.
- To explore the data using visualisations such as heatmaps, count plots and pie charts.
- To encode categorical features such as weather, meal type and day into numeric labels.
- To predict Expected_Demand using Linear Regression, Decision Tree, KNN, SVR, Random Forest and Gradient Boosting models.
- To predict Expected_Customers from weather, day, meal type, temperature, holiday and local events.
- To tune models using GridSearchCV and cross-validation, and compare them with the R^2 score.
- To save the trained models and scalers using joblib for later use.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sb

food=pd.read_csv("food_demand_expanded.csv")
```

```
food.value_counts("Weather")
```

```
Weather
Sunny     4208
Rainy     2357
Cloudy    2129
Snowy      805
Name: count, dtype: int64
```

```
food.isnull().sum()
```

```
Day                       0
Month                     0
Weekend                   0
Holiday                   0
Meal_Type                 0
Is_Payday_Week            0
Weather                 501
Temperature             499
Rainfall_mm               0
Food_Category             0
Perishability_Level       0
Price                     0
Promotion                 0
Discount_Percentage       0
Expected_Customers        0
Staff_On_Duty           501
Local_Event_Nearby        0
Previous_day_sales        0
Previous_week_sales       0
Expected_Demand           0
Prepared_Quantity         0
Expected_Waste            0
Waste_Risk                0
dtype: int64
```

```
food=food.dropna()
```

```
food.isnull().sum()
```

```
Day                     0
Month                   0
Weekend                 0
Holiday                 0
Meal_Type               0
Is_Payday_Week          0
Weather                 0
Temperature             0
Rainfall_mm             0
Food_Category           0
Perishability_Level     0
Price                   0
Promotion               0
Discount_Percentage     0
Expected_Customers      0
Staff_On_Duty           0
Local_Event_Nearby      0
```

```
Previous_day_sales     0
Previous_week_sales    0
Expected_Demand        0
Prepared_Quantity      0
Expected_Waste         0
Waste_Risk             0
dtype: int64
```

```
food["Expected_Demand"].value_counts().unique()
```

```
array([150, 147, 138, 132, 129, 127, 126, 123, 122, 120, 119, 118, 116,
       114, 113, 112, 110, 109, 103, 101,  98,  97,  95,  94,  92,  91,
        90,  87,  82,  80,  79,  78,  77,  73,  72,  71,  70,  69,  68,
        67,  66,  63,  62,  61,  60,  59,  58,  57,  56,  55,  53,  52,
        51,  49,  48,  47,  46,  45,  44,  43,  42,  41,  40,  38,  37,
        36,  35,  32,  30,  28,  25,  24,  23,  22,  20,  19,  18,  17,
        16,  15,  14,  13,  11,  10,   9,   8,   7,   6,   5,   4,   3,
         2,   1])
```

```
food["Expected_Demand"].value_counts()
```

```
Expected_Demand
67     150
71     147
73     138
82     132
79     129
      ...
202      1
165      1
168      1
211      1
209      1
Name: count, Length: 186, dtype: int64
```

```
food["Expected_Customers"].value_counts().unique()
```

```
array([93, 91, 87, 86, 85, 83, 82, 79, 78, 77, 76, 75, 74, 70, 69, 68, 65,
       64, 63, 62, 61, 60, 59, 58, 57, 54, 52, 51, 49, 48, 47, 45, 44, 43,
       42, 41, 40, 39, 38, 37, 36, 35, 34, 33, 32, 31, 30, 29, 28, 27, 26,
       25, 24, 23, 22, 21, 20, 19, 18, 17, 16, 15, 14, 13, 12, 11, 10,  9,
        8,  7,  6,  5,  4,  3,  2,  1])
```

```
food["Expected_Customers"].value_counts()
```

```
Expected_Customers
144     93
153     91
164     87
162     87
158     86
        ..
280      1
278      1
1366     1
1306     1
272      1
Name: count, Length: 299, dtype: int64
```

```
food["Meal_Type"].value_counts()
```

```
Meal_Type
Lunch         3057
Dinner        3023
Breakfast     1663
```

```
Late Night     820
Name: count, dtype: int64
```

```
food["Local_Event_Nearby"].value_counts()
```

```
Local_Event_Nearby
0    7724
1     839
Name: count, dtype: int64
```

```
food["Weather"].value_counts()
```

```
Weather
Sunny     3808
Rainy     2118
Cloudy    1907
Snowy      730
Name: count, dtype: int64
```

```
food["Weather_label"]=food["Weather"].map({"Sunny":0,"Rainy":1,"Cloudy":2,"Snowy":3})
```

```
food["Meal_Type_Label"]=food["Meal_Type"].map({"Breakfast":0,"Lunch":1,"Dinner":2,"Late Night":3})
```

```
feature_demand=food[["Expected_Customers","Previous_day_sales","Previous_week_sales","Meal_Type_Label",
"Expected_Demand","Local_Event_Nearby","Weather_label"]]
```

```
feature_demand
```

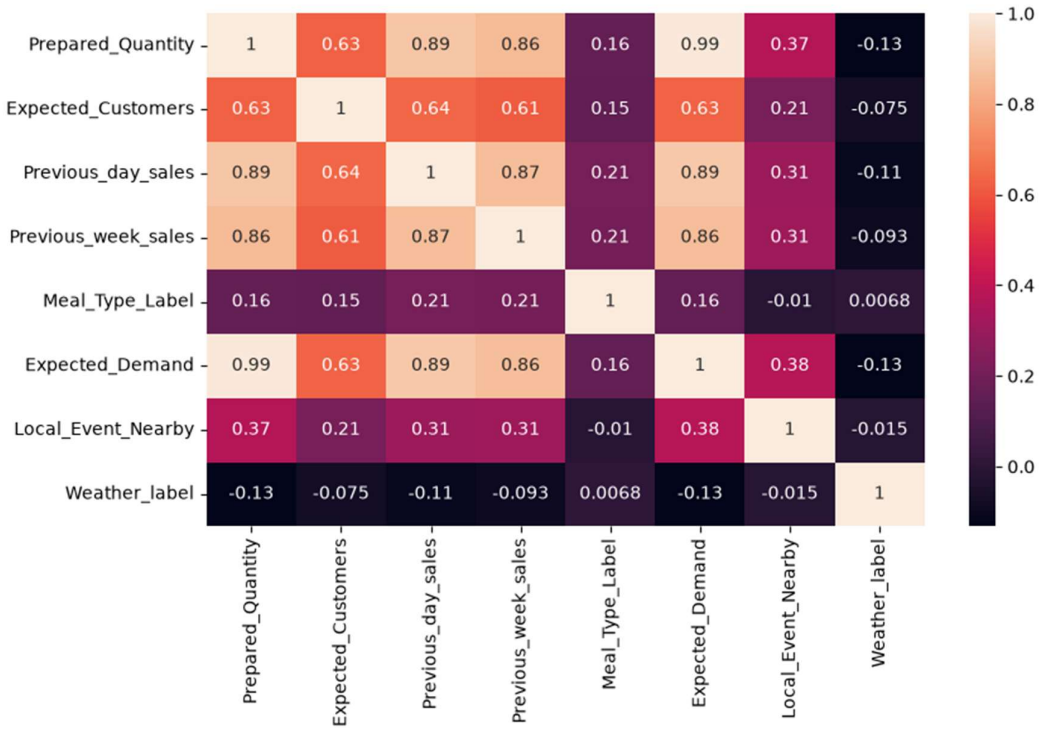
```
      Expected_Customers  Previous_day_sales  Previous_week_sales  \
0                     51                  35                   38
1                     13                   8                   10
2                    189                 155                  169
3                    127                 105                   83
4                    166                 114                  119
...                  ...                 ...                  ...
9994                  51                  32                   45
9995                 146                  99                   86
9996                  76                  54                   52
9997                 137                  88                   82
9999                 143                 102                  118

      Meal_Type_Label  Expected_Demand  Local_Event_Nearby  Weather_label
0                   3               19                   0              0
1                   3                3                   0              2
2                   1               95                   0              0
3                   1               66                   0              1
4                   2               80                   0              0
...               ...              ...                 ...            ...
9994                0               21                   0              1
9995                2               97                   0              2
9996                0               35                   0              2
9997                1               75                   0              0
9999                1               61                   0              0

[8563 rows x 7 columns]
```

```
feature_demand_corr=food[["Prepared_Quantity","Expected_Customers","Previous_day_sales","Previous_week_
sales","Meal_Type_Label","Expected_Demand","Local_Event_Nearby","Weather_label"]]
```

```
plt.figure(figsize=(9,6))
sb.heatmap(feature_demand_corr.corr(numeric_only=True),annot=True)
plt.tight_layout()
plt.show()
```




```
food.sample(5)
```

```
          Day  Month  Weekend  Holiday Meal_Type  Is_Payday_Week Weather  \
5384    Sunday      5        1        0     Lunch               0   Sunny
2047    Friday      2        0        0     Lunch               1   Rainy
8493    Friday      5        0        0     Lunch               1   Rainy
1792  Thursday      6        0        0     Lunch               1  Cloudy
4822    Sunday     11        1        0    Dinner               0   Sunny

      Temperature  Rainfall_mm Food_Category  ... Staff_On_Duty  \
5384         30.6         1.59      Beverage  ...          15.0
2047         17.3         6.23   Main Course  ...          11.0
8493         24.0        26.11       Dessert  ...           8.0
1792         22.8         1.83   Main Course  ...           7.0
4822         29.9         1.64   Main Course  ...          10.0

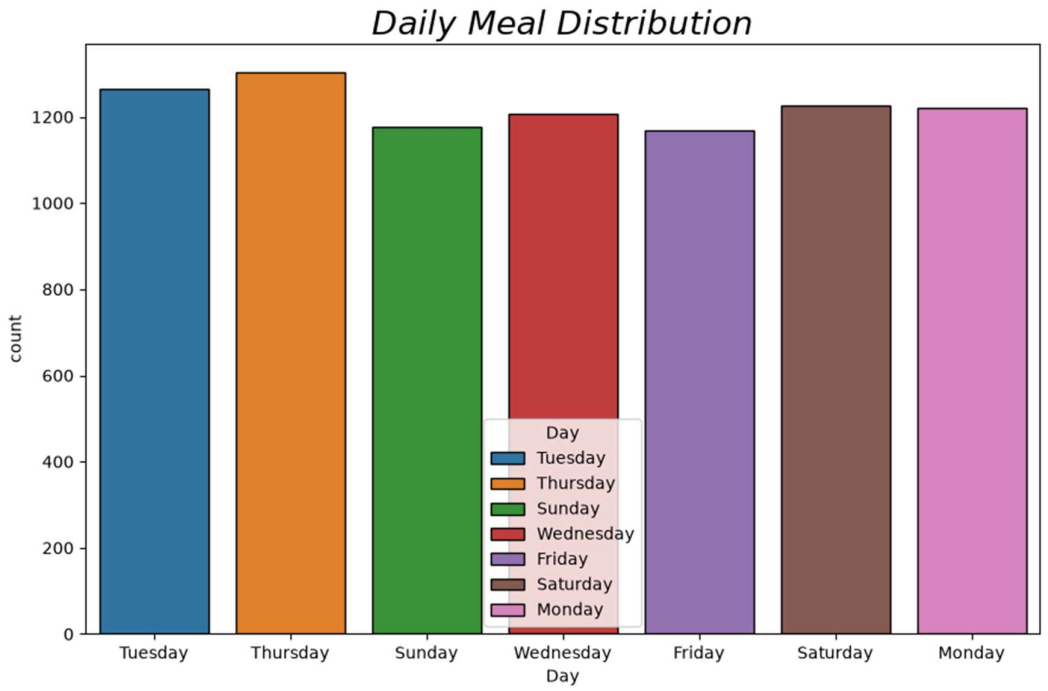
      Local_Event_Nearby  Previous_day_sales  Previous_week_sales  \
5384                   1                 177                  146
2047                   0                 102                  110
8493                   0                  66                   91
1792                   1                 161                  169
4822                   0                 123                   86

      Expected_Demand  Prepared_Quantity  Expected_Waste  Waste_Risk  \
5384              135                156              21        High
2047               80                 98              18        High
8493               54                 57               3      Medium
1792              154                180              26        High
4822               69                 74               5      Medium
```

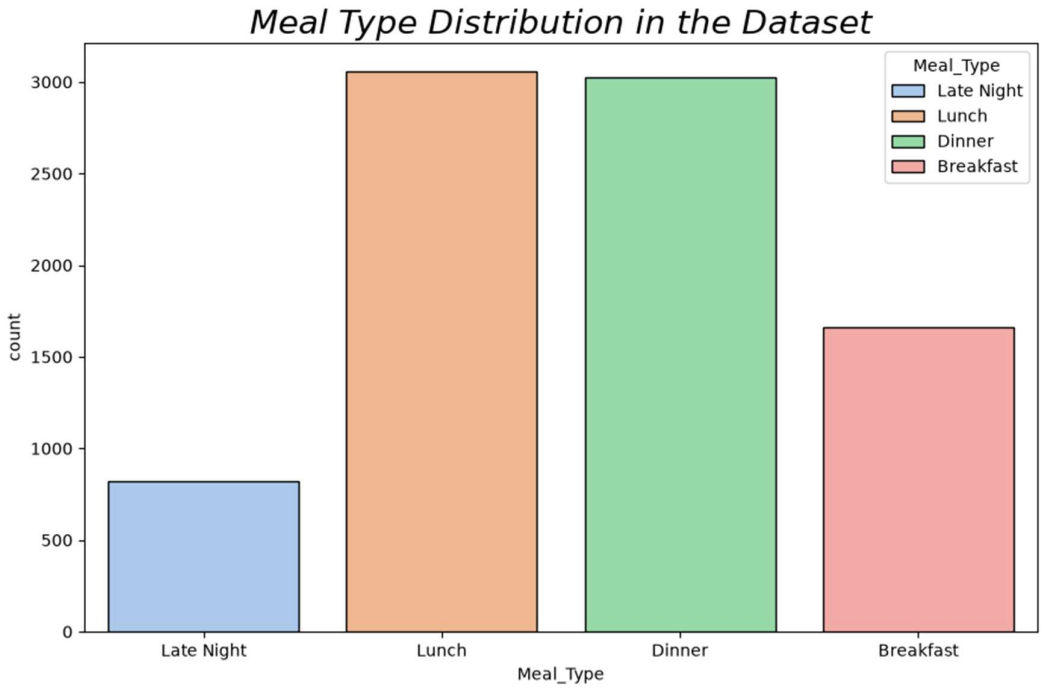
```
      Weather_label  Meal_Type_Label
5384              0                1
2047              1                1
8493              1                1
1792              2                1
4822              0                2

[5 rows x 25 columns]
```

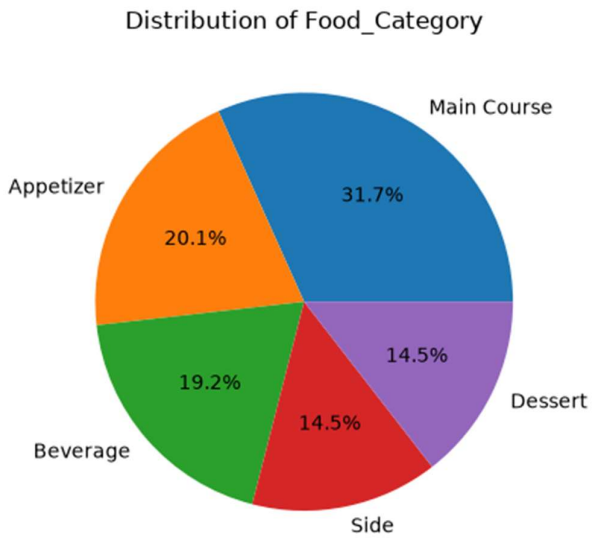
```
plt.figure(figsize=(9,6))
sb.countplot(data=food,x="Day",hue="Day",edgecolor="black",legend=True)
plt.title("Daily Meal Distribution",fontsize=20,fontstyle="italic")
plt.tight_layout()
plt.show()
```



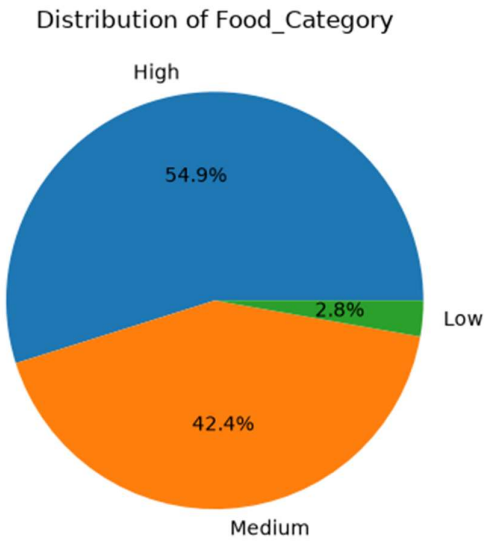

```
plt.figure(figsize=(9,6))
sb.countplot(data=food,x="Meal_Type",hue="Meal_Type",edgecolor="black",legend=True,palette="pastel")
plt.title("Meal Type Distribution in the Dataset",fontsize=20,fontstyle="italic")
plt.tight_layout()
plt.show()
```



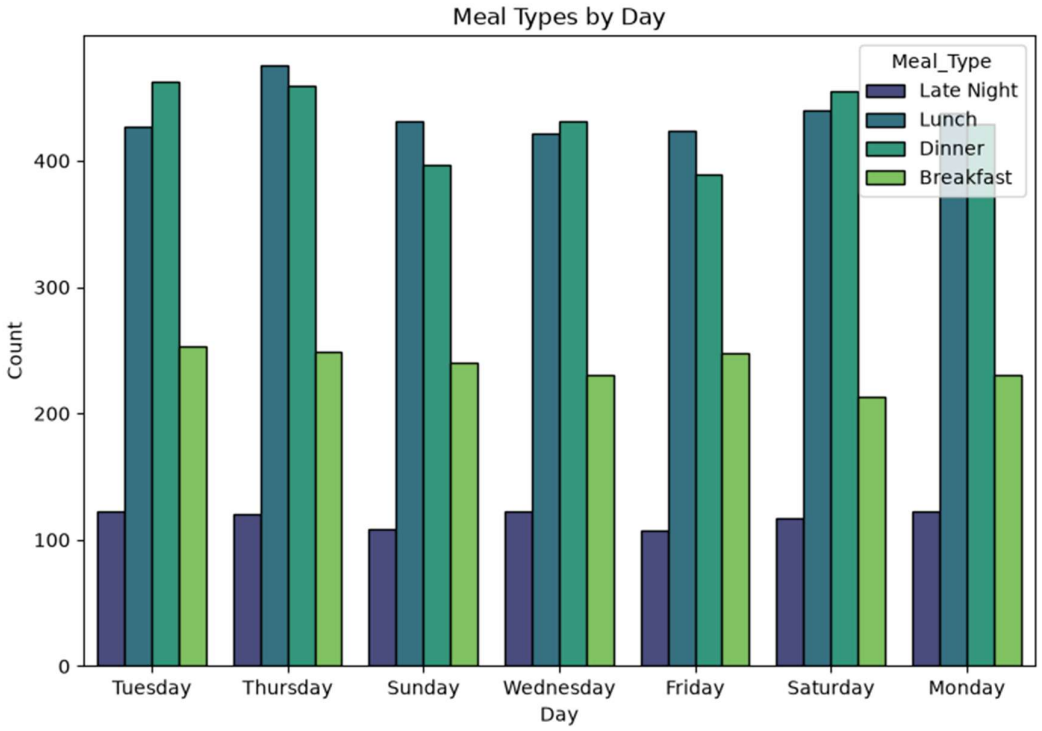

```
plt.pie(food["Food_Category"].value_counts(), labels=food["Food_Category"].value_counts().index,
autopct="%1.1f%%")
plt.title("Distribution of Food_Category")
plt.show()
```



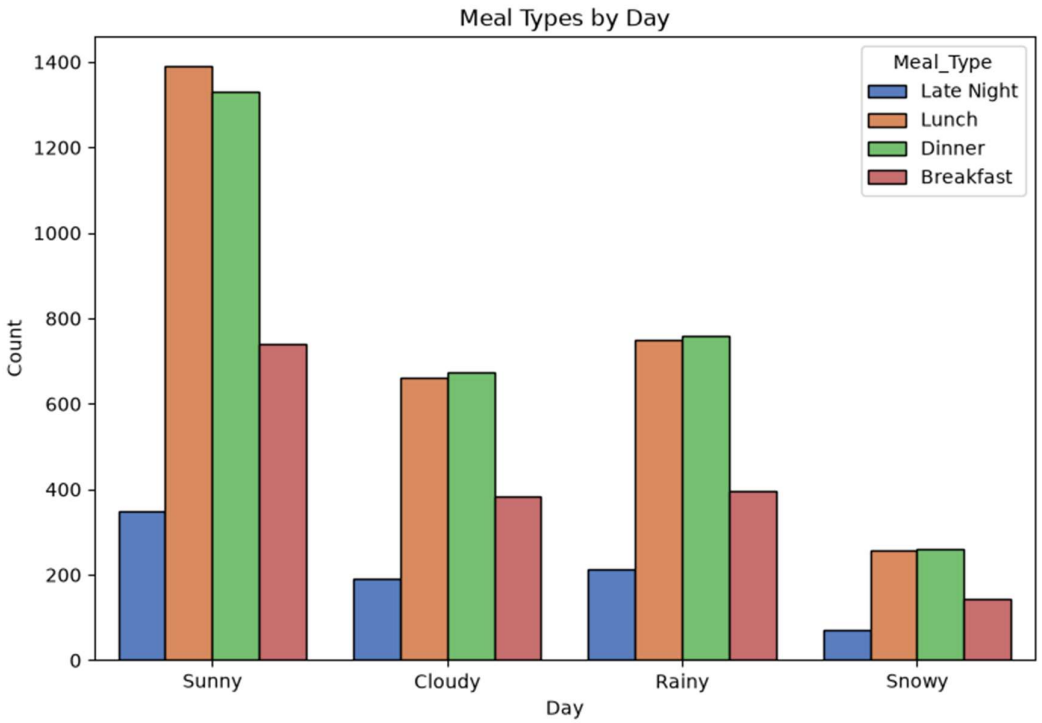

```
plt.pie(food["Waste_Risk"].value_counts(), labels=food["Waste_Risk"].value_counts().index,
autopct="%1.1f%%")
plt.title("Distribution of Food_Category")
plt.show()
```




```
plt.figure(figsize=(9,6))
sb.countplot(data=food, x="Day", hue="Meal_Type",edgecolor="black",palette="viridis")
plt.title("Meal Types by Day")
plt.xlabel("Day")
plt.ylabel("Count")
plt.show()
```




```
plt.figure(figsize=(9,6))
sb.countplot(data=food, x="Weather", hue="Meal_Type",edgecolor="black",palette="muted")
plt.title("Meal Types by Day")
plt.xlabel("Day")
plt.ylabel("Count")
plt.show()
```




```
food.columns
```

```
Index(['Day', 'Month', 'Weekend', 'Holiday', 'Meal_Type', 'Is_Payday_Week',
       'Weather', 'Temperature', 'Rainfall_mm', 'Food_Category',
       'Perishability_Level', 'Price', 'Promotion', 'Discount_Percentage',
       'Expected_Customers', 'Staff_On_Duty', 'Local_Event_Nearby',
       'Previous_day_sales', 'Previous_week_sales', 'Expected_Demand',
       'Prepared_Quantity', 'Expected_Waste', 'Waste_Risk', 'Weather_label',
       'Meal_Type_Label'],
      dtype='str')
```

Model Creation

Linear Regression model

```
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split,cross_val_score,KFold,GridSearchCV
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score,mean_absolute_error
from sklearn.tree import DecisionTreeRegressor
from sklearn.neighbors import KNeighborsRegressor
from sklearn.svm import SVR
from sklearn.ensemble import RandomForestRegressor
from sklearn.ensemble import GradientBoostingRegressor
from xgboost import XGBRegressor
from sklearn.naive_bayes import GaussianNB
from sklearn.pipeline import Pipeline

feature_demand=food[["Expected_Customers","Previous_day_sales","Previous_week_sales","Meal_Type_Label",
"Local_Event_Nearby","Weather_label"]]
st=StandardScaler()
feature_demand_st=st.fit_transform(feature_demand)
x=feature_demand_st
y=food["Expected_Demand"]
```

```
x_train,x_test,y_train,y_test=train_test_split(x,y,train_size=0.8,random_state=42)
model=LinearRegression()
training_model=cross_val_score(model,x,y,cv=5)
training_model
```

```
array([0.82678127, 0.85326976, 0.84518249, 0.84859973, 0.84010222])
```

```
print(f"your model r2_score is : \n{round(training_model.mean(),3)}")
```

```
your model r2_score is :
0.843
```

```
linear_model=LinearRegression()
model=linear_model.fit(x_train,y_train)
model
```

```
LinearRegression()
```

```
predict_model=model.predict(x_test)
print(f"your linear model r2 score :\n{r2_score(y_test,predict_model)}")
```

```
your linear model r2 score :
0.8457415132394476
```

```
feature_demand=food[["Expected_Customers","Previous_day_sales","Previous_week_sales","Meal_Type_Label",
"Local_Event_Nearby","Weather_label"]]
y=food["Expected_Demand"]
x=feature_demand
x_train,x_test,y_train,y_test=train_test_split(x,y ,train_size=0.8,random_state=42)
pip = Pipeline([
    ("tree_model", DecisionTreeRegressor(random_state=42))
])

pip_model=pip.fit(x_train,y_train)
pip_model
```

```
Pipeline(steps=[('tree_model', DecisionTreeRegressor(random_state=42))])
```

```
predict=pip_model.predict(x_test)
print(f"your decision tree regression r2_score is :\n{r2_score(y_test,predict)}")
```

```
your decision tree regression r2_score is :
0.8243330034956979
```

KNeighborsRegressor Model

```
param_grid = {"KNN_Reg__n_neighbors": [5],
    "KNN_Reg__weights": ["uniform", "distance"],
    "KNN_Reg__algorithm": ["auto", "ball_tree", "kd_tree", "brute"]}
knn_pipeline=Pipeline([("Scalar",StandardScaler()),
                       ("KNN_Reg",KNeighborsRegressor())])
model_knn=GridSearchCV(knn_pipeline,param_grid=param_grid,cv=5)
model_knn
```

```
GridSearchCV(cv=5,
             estimator=Pipeline(steps=[('Scalar', StandardScaler()),
```

```
                                ('KNN_Reg', KNeighborsRegressor())]),
             param_grid={'KNN_Reg__algorithm': ['auto', 'ball_tree', 'kd_tree',
                                                'brute'],
                         'KNN_Reg__n_neighbors': [5],
                         'KNN_Reg__weights': ['uniform', 'distance']})
```

```
model_knnreg=model_knn.fit(x_train,y_train)
model_knnreg
```

```
GridSearchCV(cv=5,
             estimator=Pipeline(steps=[('Scalar', StandardScaler()),
                                ('KNN_Reg', KNeighborsRegressor())]),
             param_grid={'KNN_Reg__algorithm': ['auto', 'ball_tree', 'kd_tree',
                                                'brute'],
                         'KNN_Reg__n_neighbors': [5],
                         'KNN_Reg__weights': ['uniform', 'distance']})
```

```
knn_predict=model_knnreg.predict(x_test)
print(f"your knn regression model r2 score is :\n :{round(r2_score(y_test,knn_predict),3)}")
```

```
your knn regression model r2 score is :
 :0.891
```

```
knn_pipeline.get_params().keys()
```

```
dict_keys(['memory', 'steps', 'transform_input', 'verbose', 'Scalar', 'KNN_Reg', 'Scalar__copy',
'Scalar__with_mean', 'Scalar__with_std', 'KNN_Reg__algorithm', 'KNN_Reg__leaf_size', 'KNN_Reg__metric',
'KNN_Reg__metric_params', 'KNN_Reg__n_jobs', 'KNN_Reg__n_neighbors', 'KNN_Reg__p', 'KNN_Reg__weights'])
```

SVR Model

```
param_grid_svr = {
    "svr__kernel": ["linear", "poly", "rbf"],
    "svr__degree": [3, 5],
    "svr__gamma": ["scale", "auto"],
    "svr__C": [0.1, 1],
    "svr__epsilon": [0.01, 0.1]
}
svr_pipeline=Pipeline([("Scalar",StandardScaler()),
                       ("svr",SVR())])
svr_model=GridSearchCV(svr_pipeline,param_grid=param_grid_svr,cv=5)
svr_model
```

```
GridSearchCV(cv=5,
             estimator=Pipeline(steps=[('Scalar', StandardScaler()),
                                ('svr', SVR())]),
             param_grid={'svr__C': [0.1, 1], 'svr__degree': [3, 5],
                         'svr__epsilon': [0.01, 0.1],
                         'svr__gamma': ['scale', 'auto'],
                         'svr__kernel': ['linear', 'poly', 'rbf']})
```

```
svr_training=svr_model.fit(x_train,y_train)
svr_training
```

```
GridSearchCV(cv=5,
             estimator=Pipeline(steps=[('Scalar', StandardScaler()),
                                ('svr', SVR())]),
             param_grid={'svr__C': [0.1, 1], 'svr__degree': [3, 5],
                         'svr__epsilon': [0.01, 0.1],
                         'svr__gamma': ['scale', 'auto'],
                         'svr__kernel': ['linear', 'poly', 'rbf']})
```

RandomForestRegressor Model

```
param_grid_rf = {
    "n_estimators": [100],
    "max_depth": [None, 10],
    "min_samples_split": [2, 5],
    "min_samples_leaf": [1, 2]
}
randomforest=RandomForestRegressor(random_state=42)
random_model=GridSearchCV(randomforest,param_grid=param_grid_rf,cv=5)
random_model
```

```
GridSearchCV(cv=5, estimator=RandomForestRegressor(random_state=42),
             param_grid={'max_depth': [None, 10], 'min_samples_leaf': [1, 2],
                         'min_samples_split': [2, 5], 'n_estimators': [100]})
```

```
random_model_training=random_model.fit(x_train,y_train)
random_model_training
```

```
GridSearchCV(cv=5, estimator=RandomForestRegressor(random_state=42),
             param_grid={'max_depth': [None, 10], 'min_samples_leaf': [1, 2],
                         'min_samples_split': [2, 5], 'n_estimators': [100]})
```

```
random_model_predict=random_model_training.predict(x_test)
print(f"your randomforest model r2 score is :\n{round(r2_score(y_test,random_model_predict),3)}")
```

```
your randomforest model r2 score is :
0.91
```

GradientBoostingRegressor

```
param_grid_gb = {
    "n_estimators": [100, 200],
    "learning_rate": [0.01, 0.05],
    "max_depth": [2, 3],
    "min_samples_split": [2, 5],
    "min_samples_leaf": [1, 2],
    "subsample": [0.8, 1.0]
}
gradient=GradientBoostingRegressor(random_state=42)
gb=GridSearchCV(gradient,param_grid=param_grid_gb,cv=5)
gb
```

```
GridSearchCV(cv=5, estimator=GradientBoostingRegressor(random_state=42),
             param_grid={'learning_rate': [0.01, 0.05], 'max_depth': [2, 3],
                         'min_samples_leaf': [1, 2],
                         'min_samples_split': [2, 5],
                         'n_estimators': [100, 200], 'subsample': [0.8, 1.0]})
```

```
gb_model_training=gb.fit(x_train,y_train)
gb_model_training
```

```
GridSearchCV(cv=5, estimator=GradientBoostingRegressor(random_state=42),
             param_grid={'learning_rate': [0.01, 0.05], 'max_depth': [2, 3],
                         'min_samples_leaf': [1, 2],
                         'min_samples_split': [2, 5],
                         'n_estimators': [100, 200], 'subsample': [0.8, 1.0]})
```

```
x
```

```
      Expected_Customers  Previous_day_sales  Previous_week_sales  \
0                     51                  35                   38
```

```
1                    13                   8                   10
2                   189                 155                 169
3                   127                 105                  83
4                   166                 114                 119
...                 ...                 ...                 ...
9994                 51                  32                  45
9995                146                  99                  86
9996                 76                  54                  52
9997                137                  88                  82
9999                143                 102                 118

      Meal_Type_Label  Local_Event_Nearby  Weather_label
0                   3                   0              0
1                   3                   0              2
2                   1                   0              0
3                   1                   0              1
4                   2                   0              0
...               ...                 ...            ...
9994                0                   0              1
9995                2                   0              2
9996                0                   0              2
9997                1                   0              0
9999                1                   0              0

[8563 rows x 6 columns]
```

```
food.columns
```

```
Index(['Day', 'Month', 'Weekend', 'Holiday', 'Meal_Type', 'Is_Payday_Week',
       'Weather', 'Temperature', 'Rainfall_mm', 'Food_Category',
       'Perishability_Level', 'Price', 'Promotion', 'Discount_Percentage',
       'Expected_Customers', 'Staff_On_Duty', 'Local_Event_Nearby',
       'Previous_day_sales', 'Previous_week_sales', 'Expected_Demand',
       'Prepared_Quantity', 'Expected_Waste', 'Waste_Risk', 'Weather_label',
       'Meal_Type_Label'],
      dtype='str')
```

```
food[["Promotion","Discount_Percentage"]].sample(5)
```

```
      Promotion  Discount_Percentage
4172          0                  0.0
9100          0                  0.0
4714          1                  6.4
6659          1                  6.3
9761          1                 18.9
```

```
food["Promotion"].value_counts()
```

```
Promotion
0    6322
1    2241
Name: count, dtype: int64
```

```
food["Discount_Percentage"].value_counts().unique()
```

```
array([6322,   18,   17,   16,   15,   14,   13,   12,   11,   10,    9,
          8,    7,    6,    5,    4,    3,    2])
```

```
food[["Staff_On_Duty","Perishability_Level"]].sample(5)
```

```
      Staff_On_Duty Perishability_Level
5886           13.0                 Low
8932           12.0                High
4550           15.0              Medium
```

```
3589           10.0              Medium
922            13.0                High
```

```
food["Perishability_Level"].value_counts()
```

```
Perishability_Level
High      3895
Medium    2965
Low       1703
Name: count, dtype: int64
```

```
food.sample(5)
```

```
           Day  Month  Weekend  Holiday  Meal_Type  Is_Payday_Week Weather  \
8167  Thursday      9        0        1      Lunch               0   Sunny
5425  Thursday      3        0        0     Dinner               0  Cloudy
4215    Monday      4        0        0     Dinner               0  Cloudy
5063    Friday      5        0        0      Lunch               1   Rainy
139   Thursday      5        0        0  Breakfast               1   Sunny

      Temperature  Rainfall_mm Food_Category  ... Staff_On_Duty  \
8167         25.2         1.93      Beverage  ...           8.0
5425         21.9         1.78   Main Course  ...           9.0
4215         24.6         1.60       Dessert  ...           7.0
5063         22.3        19.83     Appetizer  ...           9.0
139          33.0         0.96          Side  ...          10.0

      Local_Event_Nearby  Previous_day_sales  Previous_week_sales  \
8167                   1                 189                  171
5425                   0                 128                  137
4215                   0                 115                  155
5063                   0                 120                   94
139                    1                 100                   97

      Expected_Demand  Prepared_Quantity  Expected_Waste  Waste_Risk  \
8167              128                150              22        High
5425               75                 79               4      Medium
4215              120                139              19        High
5063               85                 95              10      Medium
139                57                 63               6      Medium

      Weather_label  Meal_Type_Label
8167              0                1
5425              2                2
4215              2                2
5063              1                1
139               0                0

[5 rows x 25 columns]
```

```
food["Food_Category"].value_counts()
```

```
Food_Category
Main Course    2717
Appetizer      1718
Beverage       1645
Side           1242
Dessert        1241
Name: count, dtype: int64
```

```
food["Waste_Risk"].value_counts()
```

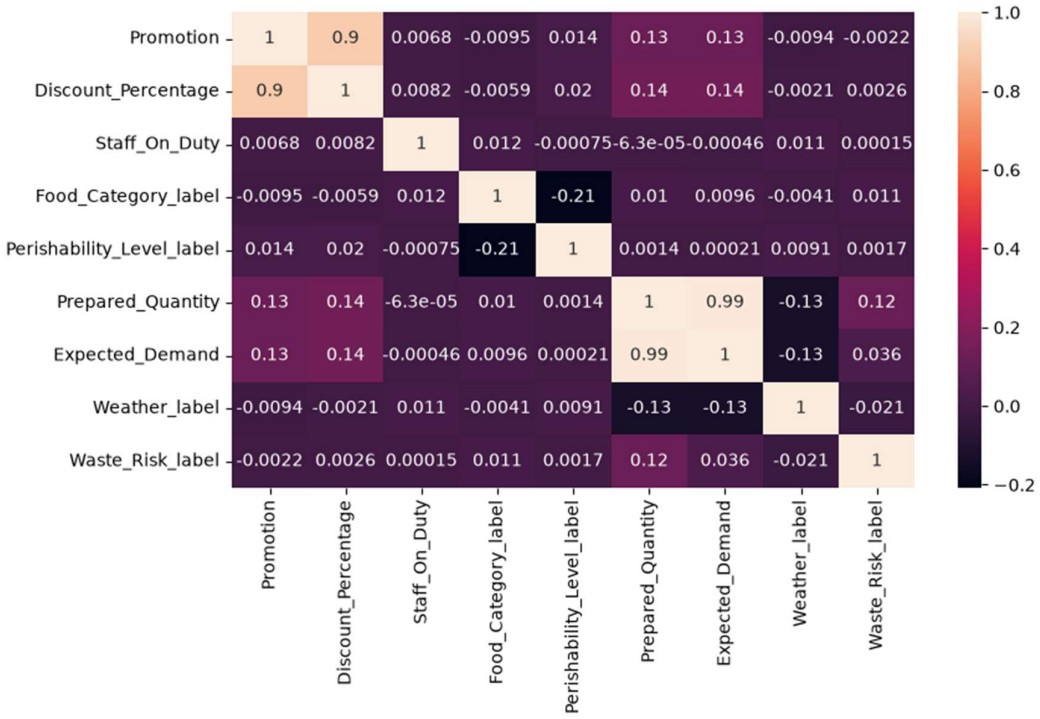
```
Waste_Risk
High      4698
```

```
Medium    3628
Low        237
Name: count, dtype: int64
```

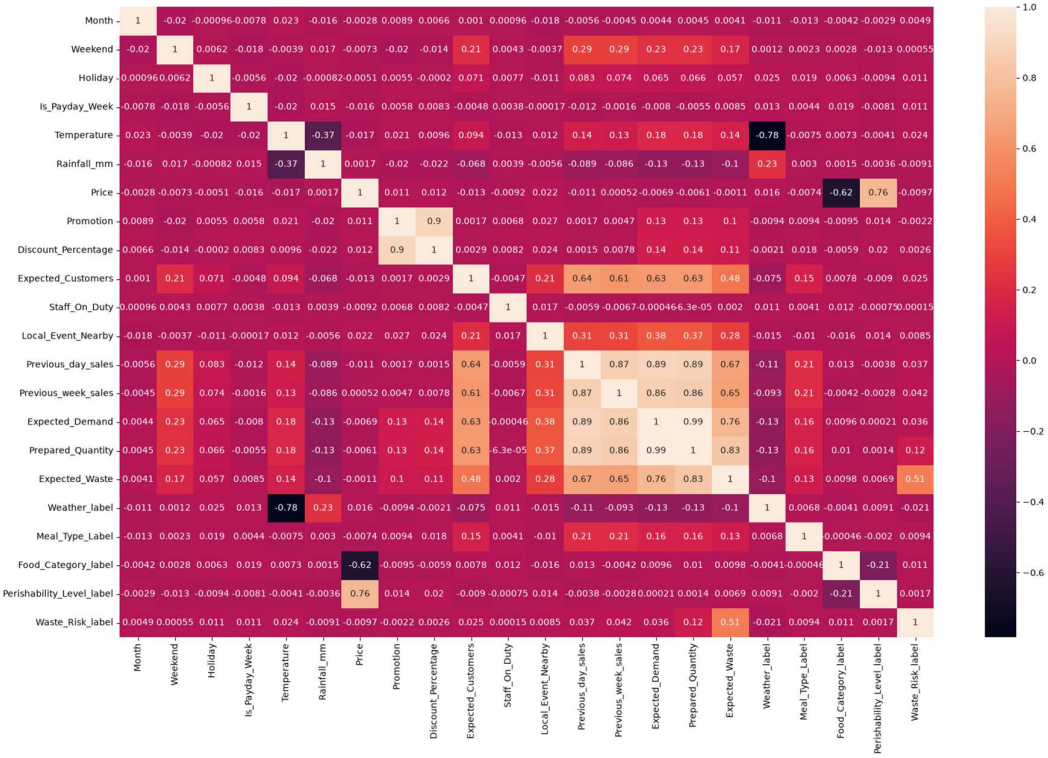
```
food["Food_Category_label"]=food["Food_Category"].map({"Main
Course":0,"Appetizer":1,"Beverage":2,"Side":3,"Dessert":4})
food["Perishability_Level_label"]=food["Perishability_Level"].map({"Low":0,"Medium":1,"High":2})
food["Waste_Risk_label"]=food["Waste_Risk"].map({"Low":0,"Medium":1,"High":2})
```

```
feature_waste=food[["Promotion","Discount_Percentage","Staff_On_Duty","Food_Category_label","Perishabil
ity_Level_label","Prepared_Quantity","Expected_Demand","Weather_label","Waste_Risk_label"]]
```

```
plt.figure(figsize=(9,6))
sb.heatmap(feature_waste.corr(numeric_only=True),annot=True)
plt.tight_layout()
plt.show()
```




```
plt.figure(figsize=(18,12))
sb.heatmap(food.corr(numeric_only=True),annot=True)
plt.tight_layout()
plt.show()
```



Expected_Customers Model

```
food["Expected_Customers"].sample(5)
```

```
3091    121
5727     87
339      89
3244     79
6820     26
Name: Expected_Customers, dtype: int64
```

```
food["Expected_Customers"].value_counts()
```

```
Expected_Customers
144     93
153     91
164     87
162     87
158     86
       ..
280      1
278      1
1366     1
1306     1
272      1
Name: count, Length: 299, dtype: int64
```

```
food["Day"].value_counts()
```

```
Day
Thursday     1303
Tuesday      1265
Saturday     1225
Monday       1220
Wednesday    1206
```

```
Sunday       1176
Friday       1168
Name: count, dtype: int64
```

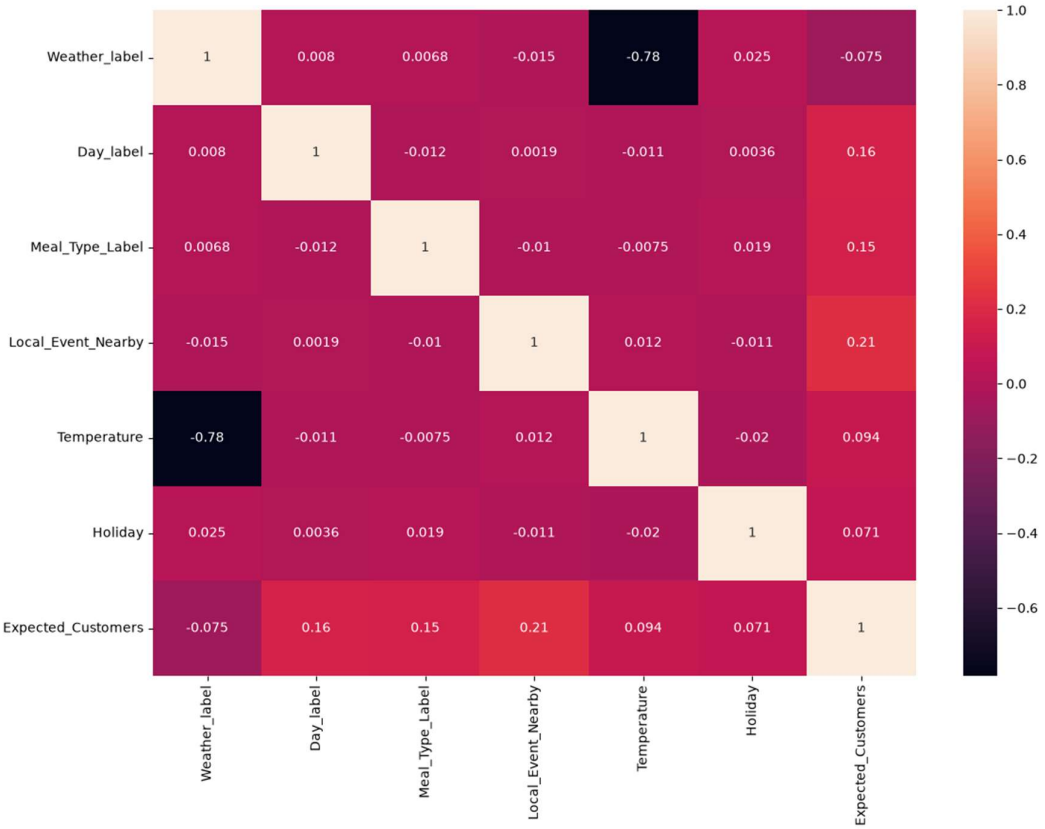
```
food["Day_label"]=food["Day"].map({"Monday":0,"Tuesday":1,"Wednesday":2,"Thursday":3,"Friday":4,"Saturday":5,"Sunday":6})
```

```
food.columns
```

```
Index(['Day', 'Month', 'Weekend', 'Holiday', 'Meal_Type', 'Is_Payday_Week',
       'Weather', 'Temperature', 'Rainfall_mm', 'Food_Category',
       'Perishability_Level', 'Price', 'Promotion', 'Discount_Percentage',
       'Expected_Customers', 'Staff_On_Duty', 'Local_Event_Nearby',
       'Previous_day_sales', 'Previous_week_sales', 'Expected_Demand',
       'Prepared_Quantity', 'Expected_Waste', 'Waste_Risk', 'Weather_label',
       'Meal_Type_Label', 'Food_Category_label', 'Perishability_Level_label',
       'Waste_Risk_label', 'Day_label'],
      dtype='str')
```

```
feature_cust=food[["Weather_label","Day_label","Meal_Type_Label","Local_Event_Nearby","Temperature","Holiday","Expected_Customers"]]
```

```
plt.figure(figsize=(12,9))
sb.heatmap(feature_cust.corr(numeric_only=True),annot=True)
plt.tight_layout()
plt.show()
```




```
food["Local_Event_Nearby"].value_counts()
```

```
Local_Event_Nearby
0    7724
1     839
Name: count, dtype: int64
```

```
feature_cust_model=food[["Weather_label","Day_label","Meal_Type_Label","Local_Event_Nearby","Temperatur
e","Holiday"]]
st_new=StandardScaler()
X=st.fit_transform(feature_cust_model)
# X=feature_cust_model
Y=food["Expected_Customers"]
X_train,X_test,Y_train,Y_test=train_test_split(X,Y)
```

```
# x_train,x_test,y_train,y_test=train_test_split(x,y,train_size=0.8,random_state=42)
model_ec=LinearRegression()
training_model_new=cross_val_score(model_ec,X,Y,cv=5)
training_model_new
```

```
array([0.1035347 , 0.10807696, 0.08313703, 0.11270606, 0.14394259])
```

```
print(f"your model r2_score is : \n{round(training_model_new.mean(),3)}")
```

```
your model r2_score is :
0.11
```

```
linear_model=LinearRegression()
model_new=linear_model.fit(X_train,Y_train)
model_new
```

```
LinearRegression()
```

```
model_pr=model_new.predict(X_test)
print(f"your r2 score is :\n{r2_score(Y_test,model_pr)}")
```

```
your r2 score is :
0.06446601253783957
```

```
param_grid_rf = {
    "n_estimators": [100],
    "max_depth": [None, 10],
    "min_samples_split": [2, 5],
    "min_samples_leaf": [1, 2]
}
randomforest=RandomForestRegressor(random_state=42)
random_model_new=GridSearchCV(randomforest,param_grid=param_grid_rf,cv=5)
random_model_new
```

```
GridSearchCV(cv=5, estimator=RandomForestRegressor(random_state=42),
             param_grid={'max_depth': [None, 10], 'min_samples_leaf': [1, 2],
                         'min_samples_split': [2, 5], 'n_estimators': [100]})
```

```
random_model_training=random_model.fit(X_train,Y_train)
random_model_training
```

```
GridSearchCV(cv=5, estimator=RandomForestRegressor(random_state=42),
             param_grid={'max_depth': [None, 10], 'min_samples_leaf': [1, 2],
                         'min_samples_split': [2, 5], 'n_estimators': [100]})
```

```
random_model_predict=random_model_training.predict(X_test)
print(f"your randomforest model r2 score is :\n{round(r2_score(Y_test,random_model_predict),3)}")
```

```
your randomforest model r2 score is :
0.26
```

```
param_grid_gb_new= {
    "estimators": [100, 200],       # Dropped 300 (can be handled by learning rate)
    "learning_rate": [0.05, 0.1],     # Dropped 0.01 (requires too many trees/time)
    "max_depth": [3, 5],              # Dropped 2 (too shallow)
    "min_samples_leaf": [1, 4],       # Kept extremes to control overfitting
    "subsample": [0.8, 1.0]           # Kept stochastic vs full sampling
}
gradient_new=GradientBoostingRegressor(random_state=42)
gb_new=GridSearchCV(gradient_new,param_grid=param_grid_gb_new,cv=5)
gb_new
```

```
GridSearchCV(cv=5, estimator=GradientBoostingRegressor(random_state=42),
             param_grid={'estimators': [100, 200], 'learning_rate': [0.05, 0.1],
                         'max_depth': [3, 5], 'min_samples_leaf': [1, 4],
                         'subsample': [0.8, 1.0]})
```

```
# gb_model_training_new=gb_new.fit(X_train,Y_train)
# gb_model_training_new
```

```
# gb_model_training_new_pre=gb_model_training_new.predict(X_test)
# print(f"your randomforest model r2 score is
:\n{round(r2_score(Y_test,gb_model_training_new_pre),3)}")
```

```
your randomforest model r2 score is :
0.445
```

```
from sklearn.datasets import make_regression



X, Y = make_regression(n_samples=1000, n_features=5, noise=0.1, random_state=42)
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state=42)



model = XGBRegressor(objective='reg:squarederror', n_estimators=100, learning_rate=0.1)
model
```

```
XGBRegressor(base_score=None, booster=None, callbacks=None,
             colsample_bylevel=None, colsample_bynode=None,
             colsample_bytree=None, device=None, early_stopping_rounds=None,
             enable_categorical=True, eval_metric=None, feature_types=None,
             feature_weights=None, gamma=None, grow_policy=None,
             importance_type=None, interaction_constraints=None,
             learning_rate=0.1, max_bin=None, max_cat_threshold=None,
             max_cat_to_onehot=None, max_delta_step=None, max_depth=None,
             max_leaves=None, min_child_weight=None, missing=nan,
             monotone_constraints=None, multi_strategy=None, n_estimators=100,
             n_jobs=None, num_parallel_tree=None, ...)
```

```
xgboost_model=model.fit(X_train, Y_train)
xgboost_model
```

```
XGBRegressor(base_score=None, booster=None, callbacks=None,
             colsample_bylevel=None, colsample_bynode=None,
             colsample_bytree=None, device=None, early_stopping_rounds=None,
             enable_categorical=True, eval_metric=None, feature_types=None,
             feature_weights=None, gamma=None, grow_policy=None,
```

```
              importance_type=None, interaction_constraints=None,
              learning_rate=0.1, max_bin=None, max_cat_threshold=None,
              max_cat_to_onehot=None, max_delta_step=None, max_depth=None,
              max_leaves=None, min_child_weight=None, missing=nan,
              monotone_constraints=None, multi_strategy=None, n_estimators=100,
              n_jobs=None, num_parallel_tree=None, ...)
```

```
predictions = xgboost_model.predict(X_test)
print(f"your model r2 score is : {round(r2_score(Y_test,predictions),3)}")
```

```
your model r2 score is : 0.95
```

```
X_train,X_test,Y_train,Y_test=train_test_split(X,Y ,train_size=0.8,random_state=42)
model_tree=DecisionTreeRegressor(random_state=42)
model_tree_fit=model_tree.fit(X_train,Y_train)
model_tree_fit
```

```
DecisionTreeRegressor(random_state=42)
```

```
predict_tree=model_tree_fit.predict(X_test)
print(f"your decision tree model r2 score is : {r2_score(Y_test,predict_tree)}")
```

```
your decision tree model r2 score is : 0.6593801242153334
```

```
param_grid_svr={"kernel":["rbf","sigmoid"],
                "degree":[3,5],
                "gamma":["scale","auto"],
                "C":[0.1],
                "epsilon":[0.01,0.1]
                }

svr_model=SVR()
svr_model_gs=GridSearchCV(svr_model,param_grid=param_grid_svr,cv=5)
svr_model_gs
```

```
GridSearchCV(cv=5, estimator=SVR(),
             param_grid={'C': [0.1], 'degree': [3, 5], 'epsilon': [0.01, 0.1],
                         'gamma': ['scale', 'auto'],
                         'kernel': ['rbf', 'sigmoid']})
```

```
training_svr_model=svr_model_gs.fit(X_train,Y_train)
training_svr_model
```

```
GridSearchCV(cv=5, estimator=SVR(),
             param_grid={'C': [0.1], 'degree': [3, 5], 'epsilon': [0.01, 0.1],
                         'gamma': ['scale', 'auto'],
                         'kernel': ['rbf', 'sigmoid']})
```

```
predict_svr=training_svr_model.predict(X_test)
print(f"your svr model r2 score is : {r2_score(Y_test,predict_svr)}")
```

```
your svr model r2 score is : 0.9999969882189003
```

```
param_grid = {"n_neighbors": [5],
    "weights": ["uniform", "distance"],
    "algorithm": ["auto", "ball_tree", "kd_tree", "brute"]}
knn=KNeighborsRegressor()
model_knn_gs=GridSearchCV(knn,param_grid=param_grid,cv=5)
model_knn_gs
```

```
GridSearchCV(cv=5, estimator=KNeighborsRegressor(),
             param_grid={'algorithm': ['auto', 'ball_tree', 'kd_tree', 'brute'],
                         'n_neighbors': [5],
                         'weights': ['uniform', 'distance']})
```

```
training_knn=model_knn_gs.fit(X_train,Y_train)
training_knn
```

```
GridSearchCV(cv=5, estimator=KNeighborsRegressor(),
             param_grid={'algorithm': ['auto', 'ball_tree', 'kd_tree', 'brute'],
                         'n_neighbors': [5],
                         'weights': ['uniform', 'distance']})
```

```
predict_knn=training_knn.predict(X_test)
print(f"your knn model r2 score is : {r2_score(Y_test,predict_knn)}")
```

```
your knn model r2 score is : 0.24940893370156825
```

```
param_grid_nb = {
    'var_smoothing': [1e-9, 1e-8, 1e-7, 1e-6, 1e-5, 1e-4, 1e-3, 1e-2]
}
nb_model=GaussianNB()
model_nb_gs=GridSearchCV(nb_model,param_grid=param_grid_nb,cv=5)
model_nb_gs
```

```
GridSearchCV(cv=5, estimator=GaussianNB(),
             param_grid={'var_smoothing': [1e-09, 1e-08, 1e-07, 1e-06, 1e-05,
                                           0.0001, 0.001, 0.01]})
```

```
training_nb=model_nb_gs.fit(X_train,Y_train)
training_nb
```

```
c:\Users\ASUS\AppData\Local\Python\pythoncore-3.14-64\Lib\site-
packages\sklearn\model_selection\_split.py:812: UserWarning: The least populated class in y has only 1
members, which is less than n_splits=5.
  warnings.warn(
GridSearchCV(cv=5, estimator=GaussianNB(),
             param_grid={'var_smoothing': [1e-09, 1e-08, 1e-07, 1e-06, 1e-05,
                                           0.0001, 0.001, 0.01]})
```

```
predict_nb=training_nb.predict(X_test)
print(f"yout naive bayes r2 score is : {r2_score(Y_test,predict_nb)}")
```

```
yout naive bayes r2 score is : -11.65823357220661
```

```
food.shape
```

```
(8563, 29)
```

```
import joblib
joblib.dump(model,"linear_reg.pkl")
joblib.dump(model_knnreg,"knn_reg.pkl")
joblib.dump(random_model_training,"random_forest_model.pkl")
joblib.dump(xgboost_model,"xgboost_model.pkl")
joblib.dump(model_tree_fit,"tree_model.pkl")
joblib.dump(st,"scaler.pkl")
joblib.dump(feature_demand.columns.tolist(), "x_columns_demand.pkl")
joblib.dump(feature_cust_model.columns.tolist(),"X_column_cust.pkl")
```

```
['X_column_cust.pkl']
```

CONCLUSION

In this project, the food demand dataset was cleaned, explored and used to build several regression models. The correlation analysis showed that Expected_Demand is strongly related to Expected_Customers, Previous_day_sales and Previous_week_sales.

For predicting Expected_Demand, the Random Forest Regressor gave the best result with an R^2 score of about 0.91, followed by the KNN Regressor (0.891), Linear Regression (0.846) and Decision Tree (0.824). This shows that demand can be predicted fairly accurately from customer counts, recent sales, meal type, weather and local events.

Predicting Expected_Customers was more difficult. Linear Regression reached only about 0.11 (cross-validated) and Random Forest about 0.26, which suggests that the available features explain customer numbers only partly and that more data or better features would be needed.

Further models (XGBoost, Decision Tree, SVR, KNN and Naive Bayes) were also tried on a separate synthetic regression dataset for comparison; their high scores there should not be read as performance on the food data. Naive Bayes, a classification algorithm, performed poorly on continuous targets, as expected.

Overall, the project shows that machine learning can support better food preparation planning and help reduce waste. The trained models were saved with joblib so they can be reused in an application in the future.

LIVE APPLICATION

The trained models were deployed as a web application named FoodSense AI, built with Streamlit, so that food demand can be predicted interactively. The live app can be opened using the link below:

Live link: https://foodsense-ai-f8qdygnspgw4zkib8jzsx2.streamlit.app/